Graduate Algorithms

Tutorial Sheet 1

August 21, 2026

Problem 1: Greatest Prime Divisor Array

For a positive integer $i \geq 2$, let $\mathrm{gpd}(i)$ denote the greatest prime divisor of i. Define an array $P[0 \ldots n]$ by

$$P[0] = 0, \qquad P[1] = 1, \qquad P[i] = \mathrm{gpd}(i) \quad \text{for } 2 \leq i \leq n.$$

(The values at 0 and 1 are fixed by convention, since neither has a prime divisor.) For example, $n = 10$ gives

$$P = [\,0,\ 1,\ 2,\ 3,\ 2,\ 5,\ 3,\ 7,\ 2,\ 3,\ 5\,].$$

We work in the following cost model. Assigning a value to a variable, adding two integers, comparing two integers, and reading or writing a single array entry given its index each cost one step. **Division and modulo are *not* assumed to cost a constant number of steps**, so you may not use them freely.

(a) Design a procedure GreatestDivisors(n) that takes a positive integer n and returns the array P, using at most $c\,n \log n$ steps for some constant c independent of n. Write it as pseudocode.

(b) Prove that your procedure returns the array P as defined above.

(c) Prove that it runs within the stated bound in this cost model.

Problem 2: Recursion Trees

Solve each of the following recurrences using a recursion tree, giving a Θ bound. In each case say which level of the tree carries the dominant cost and why. You may assume $T(a) = \Theta(1)$ for a below some constant, and you may ignore floors and ceilings.

(a) $A(n) = 5\,A(n/5) + n^2$

(b) $B(n) = B(n/4) + B(n/3) + B(n/6) + n$

Problem 3: Stooge Sort

Stooge sort is a sorting algorithm notable for being remarkably slow. To sort a segment $A[i \ldots j]$ it first compares the two end entries and swaps them if they are out of order; then, provided the segment has more than two entries, it recursively sorts three overlapping segments, each of about two-thirds the length.

Algorithm 1 Stooge sort, called as StoogeSort$(A, 1, n)$

```
1:  function StoogeSort(A, i, j)
2:      if A[i] > A[j] then
3:          swap A[i] and A[j]
4:      end if
5:      if j − i + 1 > 2 then
6:          k ← ⌊(j − i + 1)/3⌋
7:          StoogeSort(A, i, j − k)
8:          StoogeSort(A, i + k, j)
9:          StoogeSort(A, i, j − k)
10:     end if
11: end function
```

(a) Write down a recurrence for the running time of StoogeSort on an array of n entries, and solve it to show that the running time is $O\left(n^{\log_{1.5} 3}\right)$. Say which part of the recursion tree carries the dominant cost.

(b) Give an upper bound on the total number of swaps the algorithm performs.

Problem 4: Finding a Fixed Point

Let $X[1 \ldots n]$ be an array of n distinct integers, possibly negative, sorted in strictly increasing order. Call an index k a *fixed point* if $X[k] = k$.

(a) Give an algorithm that either returns a fixed point of X or correctly reports that none exists, and that is asymptotically faster than examining every entry. State and justify its running time.

(b) Now suppose you are told in advance that $X[1] > 0$. Give a *substantially* faster algorithm for the same task, and justify both its correctness and its running time.

Problem 5: Finding the Maximum and the Minimum Together

We work in the comparison model: an algorithm may compare two elements and learn which is larger, at unit cost. Given $n \geq 2$ distinct elements, the task is to report *both* the maximum and the minimum.

Prove that every deterministic algorithm for this task makes at least $\lceil 3n/2 \rceil - 2$ comparisons in the worst case.

Then show the bound is tight by exhibiting an algorithm that always achieves it.

Problem 6: A Lower Bound for Convex Hulls

Let $S = \{p_1, \ldots, p_n\}$ be a set of n points in $\mathbb{R}^d$ with $d \geq 2$. The *convex hull* of S, written $\mathrm{conv}(S)$, is the smallest convex set containing S; equivalently, it is the set of all convex combinations $\sum_i \lambda_i p_i$ with $\lambda_i \geq 0$ and $\sum_i \lambda_i = 1$. A point of S is a *vertex* of the hull if it is not a convex combination of the other points of S.

The Convex Hull problem takes S as input and must output the vertices of $\mathrm{conv}(S)$ *in boundary order*: for $d = 2$, the vertices listed in counterclockwise order around the hull.

We work in a model where an algorithm may perform arithmetic operations and comparisons at unit cost, and where the known $\Omega(n \log n)$ lower bound for sorting n numbers applies.

Prove that every algorithm for the Convex Hull problem makes $\Omega(n \log n)$ operations in the worst case, already for $d = 2$.

Problem 7: Stooge Sort, Slowed Down

Stooge sort is a sorting algorithm notable for being remarkably slow: it sorts three overlapping segments, each of about two-thirds the length. The variant below is slower still. It puts the two end entries in order, and then, on a segment of more than three entries, recursively sorts *four* overlapping segments, each of about three-quarters the length: the leading three-quarters and the trailing three-quarters, alternately, twice each.

Algorithm 2 QuadStooge, called as QuadStooge$(A, 1, n)$

```
1: function QuadStooge(A, i, j)
2:     n ← j − i + 1
3:     if A[i] > A[j] then
4:         swap A[i] and A[j]
5:     end if
6:     if n = 3 then                                   ▷ base case: three entries
7:         if A[i] > A[i + 1] then
8:             swap A[i] and A[i + 1]
9:         end if
10:        if A[i + 1] > A[j] then
11:            swap A[i + 1] and A[j]
12:        end if
13:    else if n ≥ 4 then
14:        k ← ⌊n/4⌋
15:        QuadStooge(A, i, j − k)                     ▷ leading three-quarters
16:        QuadStooge(A, i + k, j)                     ▷ trailing three-quarters
17:        QuadStooge(A, i, j − k)
18:        QuadStooge(A, i + k, j)
19:    end if
20: end function
```

(a) Write down a recurrence for the running time on an array of n entries and solve it, showing that the running time is $\Theta\left(n^{\log_{4/3} 4}\right)$.

(b) Give the best upper bound you can on the total number of swaps performed.

(c) Show that the fourth recursive call may be deleted without affecting correctness, and say what the running time becomes.